UNIDAD #2: ANÁLISIS Y DISEÑO DE SISTEMAS.

Esta unidad se divide en dos tópicos fundamentales:

2.1. Análisis de sistemas

Antes de proponer soluciones, es imperativo comprender el problema en su raíz. En esta etapa, el ingeniero actúa como un **"detective técnico"**, identificando detalles que el cliente a menudo no sabe expresar o incluso ignora. Para lograrlo, deben procesar los datos bajo dos enfoques complementarios:

- **Cuantitativo:** Se enfoca en la magnitud y frecuencia, utiliza datos numéricos y análisis estadísticos para determinar la **volumetría** del sistema (ej. cantidad de transacciones, número de registros, tiempos de respuesta requeridos).
- **Cualitativo:** Se centra en la naturaleza y atributos del fenómeno, responde al **"cómo"** y al **"por qué"** de los procesos, permitiendo entender las reglas de negocio y la experiencia del usuario.

Esto se puede determinar como la forma en la que vemos y vamos a obtener los datos que el cliente proporcione, las cuales siguen de este mismo modo, las técnicas y herramientas de elicitación empleadas en el análisis de sistemas para obtener datos de calidad y construir un sistema acorde a las necesidades del cliente y de los usuarios finales de la aplicación.

- **Técnicas de Elicitación (Recolección):**
  - **Entrevistas y guiones de entrevista:** Ideales para obtener información cualitativa directamente de los responsables del proceso.
  - **Encuestas y cuestionarios:** Herramientas masivas para recolectar datos cuantitativos y validar tendencias.
  - **Observación Directa:** Es el filtro de realidad, permite identificar procesos ineficientes que el cliente realiza por costumbre, mientras que una entrevista puede dictar que "el proceso es rápido", la observación puede revelar cuellos de botella de 20

minutos; esa discrepancia es el área de oportunidad para el ingeniero.

- **Análisis de Documentación:** revisión de facturas, reportes y archivos, aquí se encuentra la estructura de datos real que el sistema deberá persistir

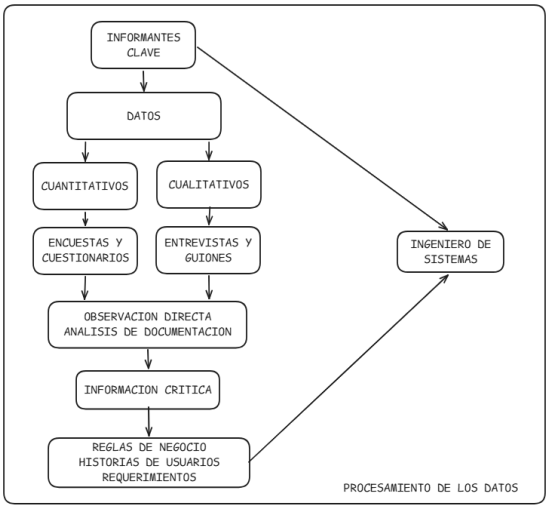
- **La Estrategia: ¿A quién preguntar?**

  La calidad de la información depende de identificar a los **Informantes Clave**, no basta con hablar con el dueño de la empresa; es vital entrevistar a quienes manejan el "monopolio de la información" en el día a día. Por lo tanto, una buena investigación requiere triangular la información: comparar diferentes puntos de vista para construir una visión objetiva, buscando deducir la información necesaria.

- **Información crítica:**

  El objetivo final del análisis es definir tres pilares que sostendrán el diseño:

  - **Entidades:** Identificación de los actores y objetos que intervienen (Usuarios, Productos, Roles).
  - **Flujos de Información:** El mapeo del proceso actual (As-Is) frente a la propuesta de mejora con el nuevo sistema (To-Be).
  - **Puntos de Falla:** Localización exacta de dónde se pierde tiempo, dinero o integridad de los datos.



2.2. Diseño de sistemas

El análisis establece el **qué** es el problema, el diseño define **cómo** será la solución. En esta etapa, el ingeniero deja de ser un **"detective técnico"** y se convierte en un **"arquitecto"**. Por lo tanto, el diseño es el "contrato" técnico que asegura que lo que se va a construir es funcional, escalable y, sobre todo, lo que el usuario necesita. Antes de trazar una sola línea de cogido, se debe establecer la lógica que gobernará el software, un diseño sin lógica es solo un dibujo que no se podrá mantener.

2.2.1. Planificación General del Proyecto (Administración de recursos)

Diseñar implica gestionar el tiempo y las herramientas, no hay ingeniería sin control de recursos.

- **Gestión del Tiempo:** Uso de cronogramas (Gantt) o tableros de gestión (Kanban) para dividir el proyecto en tareas atómicas y medibles.
    - **Objetivos:** Establecen el norte del desarrollo (**Verbo Infinitivo** (Implantar/Desarrollar) + **Producto** (Sistema de Gestión) + **Contexto** (para el Servicio de Impresiones) + **Finalidad** (para optimizar los tiempos de entrega).
        - **General:** Engloba el proceso macro (Ej: Implantar un sistema de gestión de préstamos bancarios).
        - **Específicos:** Procesos puntuales y medibles (Ej: Diseñar los elementos conceptuales del sistema).
        - **Metas:** Logro significativo que marca el fin de una etapa o la consecución de un objetivo específico (Módulo de facturación desplegado en ambiente de pruebas).
        - **Tareas:** Acción específica, técnica y medible que un desarrollador o diseñador debe ejecutar para contribuir a una meta (Diseñar el wireframe de la pantalla de Login).
- **Recurso Humano y Financiero:** Definición de roles dentro del equipo de desarrollo y estimación de costos operativos.

2.2.2. Plan de desarrollo (Lógica y Alcance)

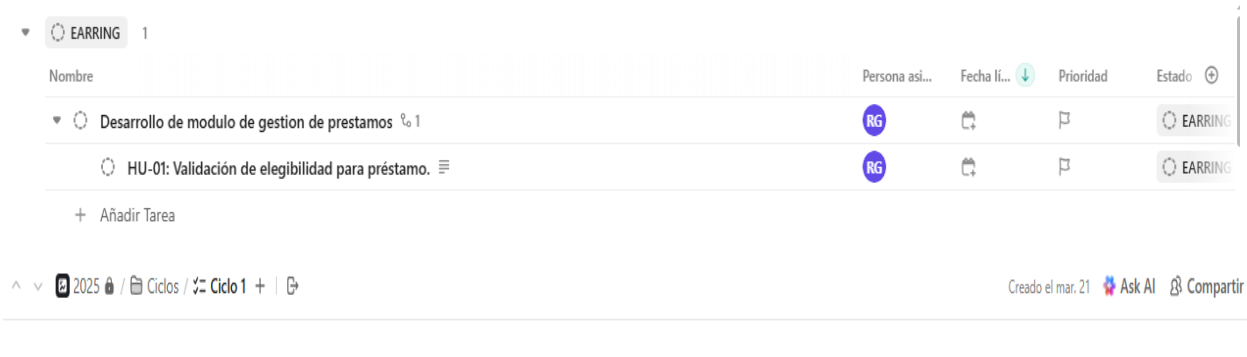
Esta etapa permite establecer el "contrato" de lo que el software hará y lo que no, reduciendo drásticamente la deuda técnica y mitigando riesgos antes de la codificación.

- **Historias de Usuario (HS):** Es la forma ágil de documentar el alcance desde la perspectiva del valor, define **qué quiere hacer** el usuario.
  - **Estructura:** "Como [rol], quiero [acción], para [beneficio]".
  - **Criterios de Aceptación:** Condiciones mínimas para dar la tarea por terminada.
  - **Ejemplo:** "**Como** cliente, **quiero** ver mi estatus de cuenta **para** saber si soy apto a un nuevo crédito".
- **Reglas de Negocio (RN):** Son las leyes del sistema, restringen las acciones de los usuarios según las políticas del mundo real.
  - **Ejemplo:** "Un usuario no puede procesar un pago si su saldo es inferior al monto total".
- **Requerimientos del Sistema (IEEE 830):** La traducción técnica de las HU y las RN.
  - **Funcionales (RF):** Acciones necesarias para que la aplicación funcione (Login, CRUD de usuarios, Generación de PDF).
  - **No Funcionales (RNF):** Atributos de calidad, no basta con que funcione; debe ser seguro, rápido y modular.

Nivel	Herramienta	¿A quién le importa?	Propósito
MACRO	**Regla de Negocio**	Al Dueño / Auditor	Definir **qué está prohibido o permitido**. Es la ley.
MESO	**Historia de Usuario**	Al Usuario / Diseñador	Definir **el flujo y el valor**. Es la experiencia.
MICRO	**Requerimiento**	Al Programador / QA	Definir **la lógica técnica**. Es la instrucción de código.

De esta forma se puede determinar, la relevancia de cada aspecto, visualizándolo desde la perspectiva de cada actor involucrado en el proceso de diseño, normalmente las reglas de negocio vienen dadas por el resultado

del análisis en la primera iteración limitando el accionar de los usuarios, la historia de usuario se constituye definiendo de forma precisa el cómo debería funcionar y por último se establecen los requerimientos funcionales/no funcionales que representan aquellos aspectos técnicos que deben tenerse en cuenta al momento de desarrollar basándose en las historias de usuario (lo que hace el sistema) y las reglas del negocio (limitantes del sistema).



HU-01: Validación de elegibilidad para préstamo.

Historia de usuario:

HU-01: Validación de elegibilidad para préstamo.

Descripción: Como cliente registrado, quiero que el sistema verifique mi estado de cuenta al intentar solicitar un crédito, para saber de inmediato si soy apto para recibir financiamiento.

1. El sistema debe consultar el estado de préstamos anteriores del usuario en la base de datos.
2. Si existe una deuda con estatus "Activo" o "Mora", el botón "Solicitar Préstamo" debe mostrarse deshabilitado.
3. Si el usuario está bloqueado, debe aparecer un mensaje explicativo: "*No cumple con los requisitos: Posee una deuda vigente*".

RN-01: Un usuario no puede procesar una nueva solicitud de préstamo si posee una deuda activa o en mora con la institución.

- **Requerimientos:**
  - **Funcionales:**
    - ~~RF-10: El sistema deberá realizar una consulta a la tabla `loan_records` filtrando por el `user_id` y el campo `status`;~~
    - RF-11: El sistema deberá bloquear el acceso al formulario de solicitud si el resultado de la consulta devuelve un registro con `status = 'ACTIVE'` o `'OVERDUE'`.
    - RF-12: El sistema deberá registrar en el log de auditoría cada intento fallido de solicitud por deuda activa.
  - **No funcionales:**
    - ~~RNF-05 (Rendimiento): La validación de deuda debe ejecutarse en un tiempo máximo de 200ms para no afectar la experiencia de navegación.~~
    - RNF-06 (Seguridad): La lógica de validación debe ejecutarse en el lado del servidor (backend), no solo en el cliente (frontend), para evitar manipulaciones del DOM.

El uso de herramientas como **ClickUp** permite integrar la estructura conceptual del análisis con la maquetación del diseño en un flujo de trabajo cohesivo. Es fundamental distinguir entre la **Planificación General del Proyecto**, que establece la hoja de ruta estratégica basada en los **objetivos específicos** (el 'qué' y 'cuándo'), y el **Plan de Desarrollo**, que se desglosa directamente desde las **Historias de Usuario** (el 'cómo' técnico). Esta distinción reduce la burocracia, ya que la planificación general dicta la gestión macro del problema desde el día uno, mientras que el plan de desarrollo actúa como una guía operativa de grano fino. Así, el ingeniero mantiene una referencia clara del progreso, transformando los requerimientos en acciones atómicas y medibles sin que la gestión se vuelva un obstáculo para la construcción del software.

2.2.3. Paquete de tecnologías

No se trata de escoger tecnologías a diestra y siniestra, la idea es escoger aquellas que permitan desarrollar las funcionalidades de la aplicación cumpliendo con las especificaciones definidas en las reglas del negocio, las historias de usuario y los requerimientos, esto se puede definir en lo siguiente:

- **Paradigma de programación:** enfoques, estilos o conjuntos de reglas que definen cómo se estructura y organiza el código para resolver problemas computacionales.
- **Arquitectura del software:** diseño de alto nivel que estructura los componentes de un sistema, sus relaciones y las reglas que los gobiernan.
- **Lenguajes de programación:** Conjunto de reglas sintácticas y semánticas que permiten a los desarrolladores escribir instrucciones para que las computadoras ejecuten tareas.
- **Librerías:** Colecciones de código prescrito, funciones y clases que facilitan el desarrollo de software.

- **Frameworks:** Estructura conceptual y técnica predefinida que proporciona herramientas, bibliotecas y estándares para agilizar el desarrollo de software.
- **Patrones de diseño:** Son soluciones habituales a problemas comunes en el diseño de software.
- **Base de datos:** Herramienta digital que permite organizar, almacenar y gestionar grandes volúmenes de información de forma estructurada y segura.
- **Gestor de base de datos:** Son software que permiten crear, mantener y administrar bases de datos, actuando como interfaz entre los datos, los usuarios y las aplicaciones
- **El mapeo objeto-relacional (ORM):** Es el proceso de abstraer la conexión entre las entidades (objetos) del lenguaje de programación y sus correspondientes elementos de base de datos.
- **Hosting:** Servicio que alquila espacio en un servidor físico o virtual, conectado 24/7 a internet, para almacenar los archivos, imágenes y datos de una página web.
- **Control de versiones:** es un sistema que registra los cambios en archivos de software a lo largo del tiempo, permitiendo recuperar versiones anteriores, trabajar en equipo simultáneamente y gestionar el historial del proyecto.
- **Repositorio:** Espacio centralizado, generalmente digital, donde se almacenan, organizan, gestionan y preservan datos, archivos o códigos de forma ordenada.

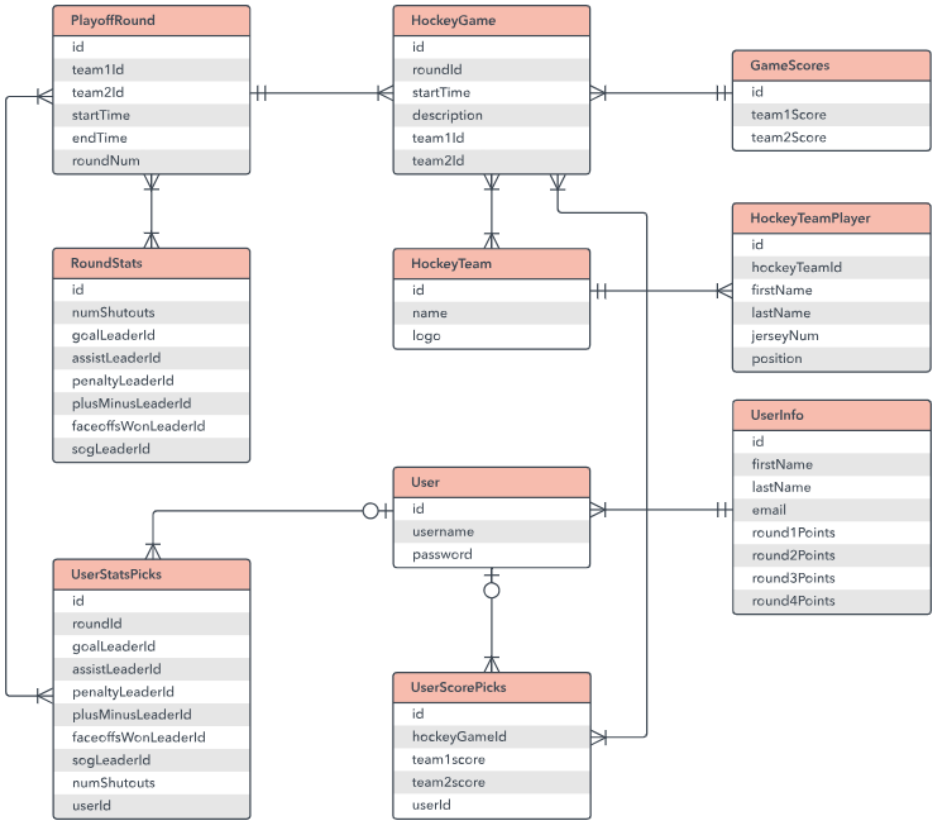
La selección del stack no es un gusto personal, es una decisión de ingeniería basada en el rendimiento, la seguridad y el tiempo de entrega.

2.2.4. Modelado Técnico (La Estructura Interna)

Aquí el diseño se vuelve técnico y se prepara para el desarrollo.

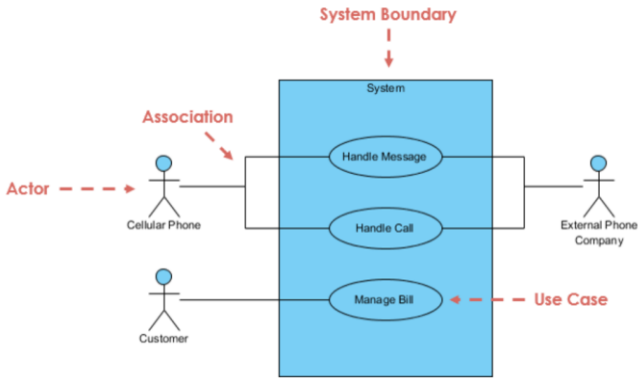
- **Diseño de Base de Datos:**

- **Modelo Entidad-Relación (ER):** El mapa de cómo se almacenará la información.

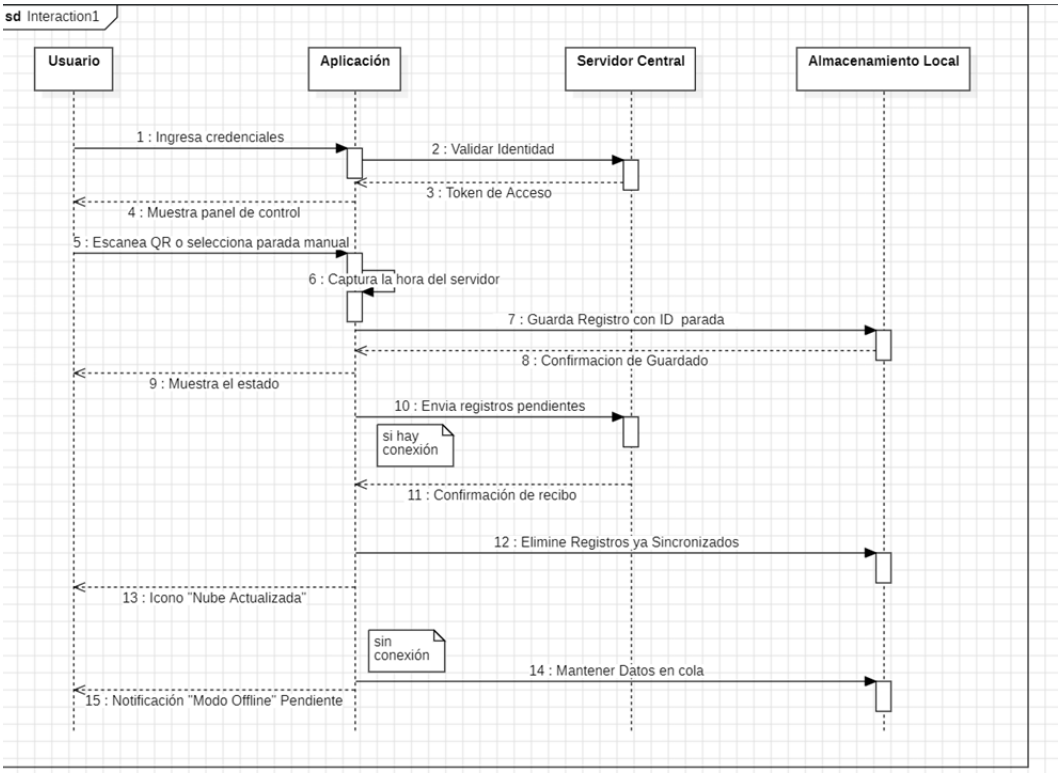


- **Diccionario de Datos:** La especificación técnica de cada campo (Nombre, Tipo de dato, Longitud, Restricciones).

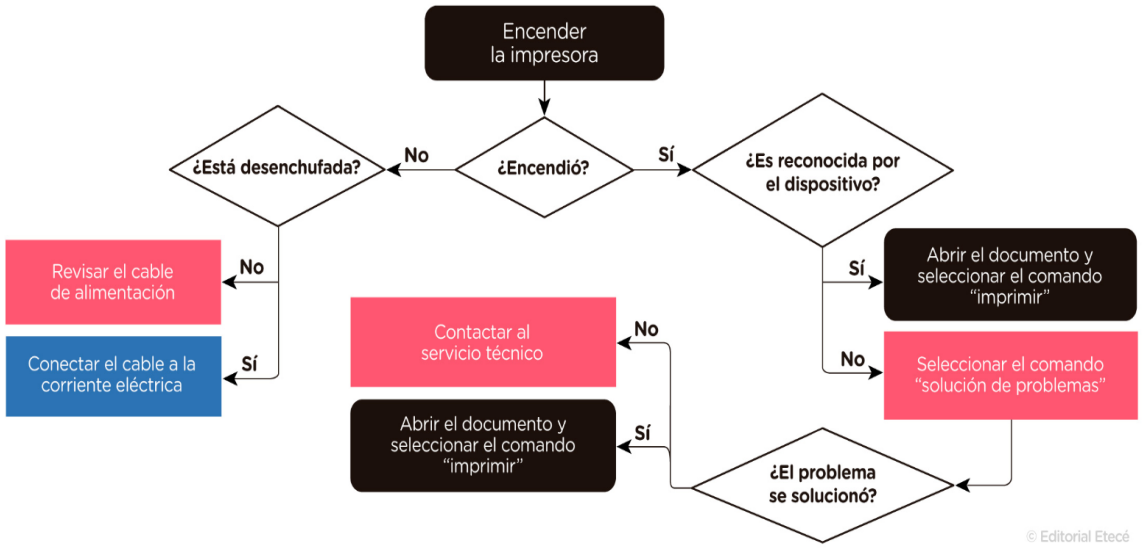
- **Diagramas UML (Unified Modeling Language):**
  - **Diagrama de casos de uso:** permiten al ingeniero visualizar como seria la interacción de los tipos de usuarios en diferentes contextos dentro de la aplicación.



- **Diagrama de Secuencia:** Muestra cómo interactúan los objetos en el tiempo, Indispensable para entender el flujo entre el Frontend y el Backend, permitiendo la visualización de toda la aplicación que se pretende desarrollar.



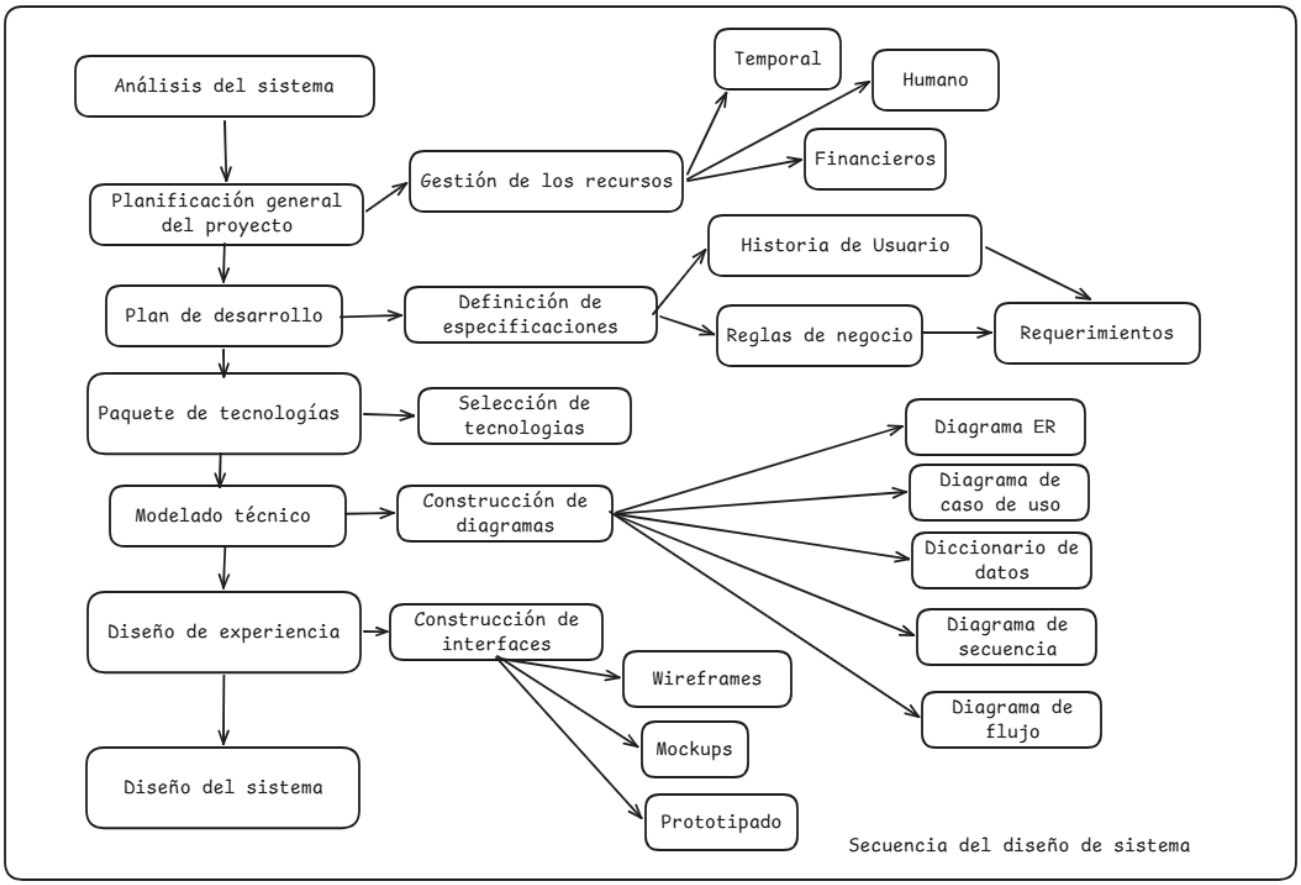
- **Diagrama de Flujo:** Define la lógica de procesos complejos o algoritmos crítico.



2.2.5. Diseño de Experiencia e Interfaz (UX/UI).

Es infinitamente más barato corregir un boceto que corregir 1,000 líneas de código, por lo cual, una vez definido los aspectos técnicos de la aplicación, se recomienda preparar un diseño preliminar de cómo se podría ver, integrando aspectos visuales y funcionalidades para que el cliente pueda dar sus observaciones antes de siquiera desarrollar una línea de código, por lo que empleamos tres técnicas para poder presentarlo.

- **Wireframes (Baja Fidelidad):** Es el esqueleto. Aquí no importan los colores ni el logo; importa la **jerarquía de la información** y la ubicación de los elementos. Sirve para validar la navegación.
- **Mockups (Alta Fidelidad):** Es la representación visual final. Aquí se define la identidad visual (UI) que el cliente verá.
- **Prototipado:** Es la simulación de flujo. Permite "navegar" por el diseño antes de que exista el backend.



Secuencia del diseño de sistema